

Project Brief: Backstop — Runtime Failure Prediction for Vision-Language-Action Robot Policies

Prepared for: Dr. Benjamin M. Drury, ChiEAC **Submitted by:** Gautham Ramkumar
Date: August 2026 **Build window:** 6 weeks to public MVP **Capital budget:** \$45 (ceiling \$120) **Status:** Proposed

1. Executive Summary

Vision-Language-Action models are the dominant approach in robot manipulation right now. A single neural network takes camera images and a natural-language instruction and emits robot actions directly. They generalize impressively, and they fail silently. The policy does not know it is about to knock over the cup, close the gripper on empty air, or confidently execute a task whose target object was never in the scene. It emits the same well-formed action stream either way.

Backstop is the missing supervisory layer: a runtime monitor that predicts execution failure *before* the arm commits to it, and halts or requests help instead of completing a destructive motion. It does not replace the policy. It watches the policy and decides whether to trust the next action chunk.

The system is built and evaluated on LIBERO, the standard open benchmark for language-conditioned manipulation, using publicly released VLA checkpoints. The deliverable is a hosted, interactive rollout viewer where any visitor can replay an episode, watch the confidence signals evolve step by step, and see exactly where the guard fired relative to where the failure actually occurred.

The claim being tested: failure is predictable from signals the policy already produces, without retraining the policy and without privileged simulator state. If true, this is a drop-in safety layer for any VLA deployment. If false, that is also a reportable result, and the failure analysis is the contribution.

2. Problem Statement

The operator's problem

A warehouse or lab deploying a VLA policy today has a binary choice: run it autonomously and absorb the cost of silent failures, or keep a human watching every episode, which erases the reason for deploying it. There is no middle option, because there is no signal that says "this particular episode is going wrong." Success rates are reported per-benchmark in aggregate; nothing tells you about the run currently executing.

Why this is structurally hard

A VLA's output distribution is not calibrated to task success. The model is trained by imitation on successful demonstrations, so it has never been supervised on what failure looks like. It produces confident actions in states it has never seen. Standard classifier uncertainty methods do not transfer cleanly, because the output is a continuous action sequence, not a class label, and because the relevant failure is temporal — it unfolds over a chunk of future actions, not a single step.

Why the timing is right

Until recently there were no open VLA checkpoints to study. There now are, along with a standard benchmark, published baseline success rates, and a research community actively naming runtime monitoring as an open problem. The prerequisites for this work exist and did not eighteen months ago.

Consequence

Every VLA deployment currently ships without a supervisor. The engineering layer that makes these policies operationally safe is the part nobody has built.

3. Project Definition

3.1 The core system

Observation and action stream in, per-step trust score out, with a halt decision when the score crosses a calibrated threshold.

3.2 Signals the guard uses

Signal	What it detects
Action-chunk disagreement	Sample the policy multiple times under stochastic decoding; measure divergence across sampled chunks. High spread means the policy has no confident opinion about what to do next.
Temporal self-consistency	Chunks predicted at successive timesteps overlap in the actions they cover. When the overlapping region disagrees, the policy is changing its mind mid-motion — a

strong precursor to failure.

Visual precondition check	Before committing to a chunk, verify the instructed object is actually present and reachable. Grounded in prior work where a vision-language model authorized trajectories toward objects that were not in the scene.
Progress monitoring	Compare predicted state change against observed state change. A gripper that closes with no corresponding visual change has grasped nothing.

Each signal is evaluated independently first, then combined into a single trust score by a lightweight learned head trained on recorded rollouts. Reporting the per-signal ablation matters as much as reporting the combined number, because it says which signal is doing the work.

3.3 What ships

Deliverable	Purpose
Interactive rollout viewer	Hosted, public URL. Replay any recorded episode with synchronized video, per-step trust score, guard-fire marker, and ground-truth outcome.
Evaluation report	Detection rate at fixed false-stop budgets, detection lead time, per-signal ablation, per-task-suite breakdown.
Failure taxonomy	Categorized catalogue of observed failure modes with representative episodes for each. Useful independent of whether the guard works.
Reproducible pipeline	Containerized, CI-benchmarked, rollout recording and evaluation runnable from one command.

3.4 The metric that actually matters

Raw accuracy is the wrong measure. A guard that halts everything catches every failure and is useless. The operational question is: *at a false-stop rate an operator would tolerate, what fraction of failures does the guard catch, and how many steps of warning does it give?* Headline result is reported at a fixed false-stop budget, with the full trade-off curve published alongside it.

4. Technical Architecture

Policy checkpoint → LIBERO rollout with perturbation injection → recorded episodes with per-step action distributions → offline guard training and evaluation → exported artifacts → web viewer

Platform

LIBERO, built on MuJoCo, is the right substrate. It is the benchmark published VLA results are reported against, so baseline numbers are directly comparable rather than self-defined. It is light enough to run on free-tier cloud GPU, unlike photorealistic alternatives that would consume weeks in setup before producing a single measurement.

Policy selection

Start with a small open checkpoint sized to run comfortably in a free-tier session; scale up to a larger open VLA only after the recording and evaluation loop is proven. The guard is policy-agnostic by construction — it consumes action distributions and observations, not model internals — so the choice of checkpoint is an implementation detail rather than a dependency.

Generating failures on purpose

The project needs many failures with unambiguous ground-truth labels. Simulation is not a compromise here; it is the enabling condition. Perturbations are injected systematically: object pose offsets, distractor objects, lighting variation, instructions referring to absent objects, and action noise. Each perturbation is labeled by construction, which is precisely what hardware cannot provide at this volume.

Decoupling the expensive part

Rollout generation is slow and runs once. Guard training and evaluation run against recorded episodes and iterate in seconds. Recording to disk early is the difference between a project that iterates and one that waits on the GPU all six weeks.

Infrastructure

Free-tier cloud GPU notebooks for rollout generation and training, with the benchmark data hosted rather than downloaded. Docker containerization and GitHub Actions CI with automated evaluation benchmarks, following the pattern established in prior work. Only the exported viewer assets and frontend live locally.

5. Scope and Cut Lines

In scope for MVP:

- Two LIBERO task suites, covering spatial reasoning and object generalization
- One policy checkpoint, evaluated end to end
- All four guard signals implemented and independently ablated
- Hosted rollout viewer with the full recorded episode set
- Published evaluation report with trade-off curves

Explicitly deferred:

- Real hardware validation
- Recovery behaviour — the guard halts and flags; it does not replan
- Policy fine-tuning or retraining of any kind
- Multiple policy architectures compared against each other
- Long-horizon and multi-stage task suites

Hard cut line if week 4 runs late: ship the failure taxonomy and the single strongest individual signal, with the rollout viewer, and drop the learned combination head. A categorized public catalogue of VLA failure modes with a working detector for one of them is a genuine contribution on its own. The combined trust score is the impressive part, not the necessary part, and it can land two weeks after launch.

6. Week-by-Week Plan

Week 1 — Rollout infrastructure. Stand up LIBERO with a policy checkpoint. Reproduce a published baseline success rate to confirm the setup is correct before building anything on top of it. Record episodes to disk with per-step action distributions and observations. *Exit criterion:* a reproduced baseline number within tolerance of published results, and a stored episode set.

Week 2 — Failure generation and taxonomy. Implement perturbation injection. Generate a balanced set of successful and failed episodes across perturbation types. Manually review and categorize failure modes. *Exit criterion:* a labeled episode corpus with enough failures per category to evaluate against, and a written taxonomy.

Week 3 — Signals. Implement all four guard signals offline against recorded episodes. Evaluate each independently. *Exit criterion:* per-signal detection curves, and an honest statement of which signals carry information and which do not.

Week 4 — Combination and calibration. Train the trust-score head. Calibrate thresholds against false-stop budgets. Measure detection lead time. Run held-out evaluation on a task suite never used during development. *Exit criterion:* headline result at a fixed false-stop rate, on data the guard has not seen.

Week 5 — Web delivery. Build the rollout viewer: synchronized episode playback, live trust-score trace, guard-fire marker, ground-truth outcome, filterable by failure category. Test on mobile over cellular. *Exit criterion:* a public URL where a visitor understands the result in under sixty seconds without reading anything.

Week 6 — Write-up, outreach, and buffer. Publish the evaluation report and failure taxonomy. Reserve capacity for whatever broke in week 1.

7. Budget

Item	Cost	Notes
Cloud GPU — rollout generation and training	\$0	Free-tier notebook quota is sufficient at this scale
Overflow GPU capacity	\$35	Spot instances if free quota is exhausted during rollout generation
Hosting — viewer and episode assets	\$10	Static hosting; served from existing portfolio domain
Subtotal	\$45	
Contingency	\$75	Additional rollout runs, storage for expanded episode set
Total ceiling	\$120	

Cost discipline. Recording rollouts once and iterating offline is what keeps GPU spend near zero. Every hour of guard development that runs against stored episodes instead of live simulation is an hour of GPU time not purchased. Treat the recording step as a cost control, not just an engineering convenience.

8. Public Deliverable

The live URL is an interactive rollout viewer hosted as a subpath of the existing portfolio site, so that traffic to the demo compounds into visibility for the full body of work rather than scattering across platforms.

A visitor selects an episode. Video plays. A trust-score trace runs beneath it in sync. A marker shows where the guard fired. A second marker shows where the failure actually occurred. The gap between them is the entire result, and it is legible without a caption.

Episodes are filterable by outcome and failure category, so a technical visitor can go directly to the cases where the guard was wrong. Showing the false stops and the misses deliberately is not a concession — it is the thing that distinguishes an engineering artifact from a marketing one. The underlying repository is linked prominently from the viewer for reviewers who want to read code.

9. Key Risks

Risk	Severity	Mitigation
Policy checkpoint too heavy for free-tier memory	High	Start with a small open VLA; quantize if needed. The guard is architecture-agnostic, so this changes the baseline number and nothing else.
Signals carry no predictive information	High	Evaluate each signal independently in week 3, before investing in combination. A negative result reported honestly, with the failure taxonomy, is still a publishable deliverable.
Failure rate too low to evaluate against	Medium	Perturbation strength is a tunable knob. Sweep it in week 2 until the corpus is balanced.
Simulation-only results invite a transfer objection	Medium	State the limitation explicitly and name which signals are most likely to degrade on hardware. Preempting the question converts the weakest point into a credibility signal.
Viewer payload too heavy for mobile	Low	Precompute and compress episode assets; test on a real mid-range device over cellular in week 5, not on a laptop.

On the second risk: it is worth being direct that this is a research question with a real possibility of a negative answer. That is why the failure taxonomy is scoped as an independent deliverable in week 2 rather than a byproduct of week 4. The project produces something of value even in the branch where the central hypothesis does not hold.

10. Alignment with Program Requirements

Requirement	How this project meets it
Aligns with target role in 6–12 months	Runtime safety and reliability engineering for learned robot policies is an explicitly named function at robotics companies deploying manipulation systems.
Strengthens a developing skill, not a repeat of prior work	No prior project involves VLA policies, uncertainty estimation, or runtime monitoring. The closest prior work is a hand-built fallback ladder on physical hardware — this is the systematic, measured version of an intuition that was previously implemented by hand.
End to end, resulting in a live URL	Rollout generation through hosted interactive viewer, publicly accessible without installation.
Relevant to hiring trends in the next six months	Vision-language-action models are the current center of gravity in robot manipulation. Their reliability gap is openly acknowledged and unsolved.

11. Career Positioning

The resume line: Built and shipped a runtime failure-prediction system for vision-language-action robot policies, detecting execution failures before commitment at a calibrated false-stop budget, with a published failure taxonomy and a public interactive evaluation viewer.

Three reasons this is strong. It targets a named open problem rather than reimplementing a solved one, which is the difference between a portfolio project and a coursework artifact. It is safety-critical work with an explicit cost asymmetry, framed the way reliability engineers actually frame decisions. And the deliverable is an evaluation surface rather than a demo — showing where the system fails is a more credible engineering signal than showing only where it succeeds.

It also widens rather than narrows. The underlying competencies — uncertainty estimation, runtime monitoring, evaluation design under asymmetric cost — apply directly to autonomous vehicle perception, industrial inspection,

and any deployed learned system where silent failure is expensive.

12. Immediate Next Steps

1. Stand up the benchmark and reproduce a published baseline success rate before writing any guard code. If the baseline does not reproduce, everything measured afterward is unreliable.
2. Record and store rollouts to disk in week 1, ahead of schedule if possible. Every downstream week depends on iterating offline rather than regenerating episodes.
3. Manually watch fifty failed episodes before designing any signal. The taxonomy should come from observed failures, not from theory about what failure ought to look like.
4. Fix the false-stop budget and the headline metric definition in week 1, in writing, before any number exists. Choosing the metric after seeing results is how projects end up reporting the flattering number instead of the useful one.